

New York University — Tandon School of Engineering
 Department of Technology Management and Innovation
IE-GY 9113B — Systems Integration: From ERP to Agentic AI
Study Note — Week 2: Failure to Leverage + BPR as Remedy
 Module 1: The Integration Foundation | Friday, September 11, 2026

Readings for this week	
Primary case	Birmingham City Council / Oracle (2022–23) — read the council’s Section 114 statement (birmingham.gov.uk/.../section_114) against the University of Sheffield Audit Reform Lab report (auditreformlab.shef.ac.uk, PDF). Read the Section 114 statement in full (about a page); in the Audit Reform Lab report, focus on the executive summary and the sections on the Oracle implementation and its cost, and skim the rest. Read these together so you see the causal dispute first-hand. A third primary source, the external auditor’s report (Grant Thornton), birmingham.gov.uk/.../external-audit, is worth a skim for the governance verdict; budget about an hour in total.
Supporting	Nike i2 supply-chain failure (2000) — Koch, “Nike Rebounds” (CIO, 2004): cio.com/.../nike-rebounds. Henrico Dolfing’s write-up is a further supporting reference: henricodolfing.ch (Case Study 16).
Go deeper (optional)	This note teaches these directly, so the books are optional deepening. For the full treatment: Ross & Weill, <i>Enterprise Architecture as Strategy</i>, Ch. 2–3 (operating models; enterprise architecture); and Hammer’s 1990 HBR essay, “Reengineering Work: Don’t Automate, Obliterate” (HBR reprint 90406; via NYU Libraries), the origin of the rule this week teaches.
Before Lab 1 (Wk 3)	Lab 1 pre-work is assigned this week and spans Weeks 2–3: pick your company, research its real friction and value leakages from public sources through the Four Lenses, and draft a ½–1 page research brief — submitted and checked before the Week 3 lab. Your Week 1 sandbox (Airtable or Google Sheets with Gemini) is ready for the build; the team charter is also due end of Week 2.

Week 1 ended on a single, unsettling idea, and this week is built to sit inside it. We closed by naming **the leverage gap** — the distance between a backbone that works and a backbone that pays — and promised that Week 2 would dissect it. This is that dissection. You already have the floor beneath you: Week 0 (Course Primer) set the ERP floor and taught you to read a process, and Week 1 defined the integration backbone, its single source of truth, its control points, and the operating model that shapes it. Now we ask the question that the rest of your career will keep asking: when the system is in place and even works, why does the value so often fail to arrive — and what do you do about it?

Hold the frame first, because everything here hangs off it. The course runs along one arc — **integrate, automate, make intelligent, make agentic** — and Module 1 is the first word in it. Three threads run through everything: the **Customer** thread (taking an order and serving the person as one coherent promise), the **Operations and Supply Chain** thread (planning, inventory, and synchronization, so a company can deliver what it promised), and the **Cash Management** thread (how fast and how reliably the backbone turns activity into cash — the thread that matters because cash flow is the most common way businesses fail). We keep four archetypes in view the whole way: a consumer-packaged-goods manufacturer running sales-and-operations planning on SAP; a mid-market manufacturer running order-to-cash on NetSuite; a Series-B startup running order-to-cash across Stripe, Shopify, and QuickBooks; and a fifty-person services firm living in HubSpot and Notion. And we read every company through the operating-model lens — standardization and integration, and the four stances they produce.

Here is what this week delivers, in order. First, the distinction the whole week turns on: failure to leverage versus failure to implement. Then the reason integration is so unforgiving in practice — the delivery ecosystem and the legacy, multi-system reality that make real backbones so hard to change, and why the requirements and partner decisions you make early decide the outcome. Then the cure: redesign the process before you automate it — “obliterate, then automate” — but selectively, concentrating redesign where value is created or destroyed rather than reshaping the business to fit the platform — and a disciplined way to find those few areas, the Four Lenses, now in full. Two cases a generation apart, Birmingham and Nike, show the identical trap. And finally the human precondition that makes the cure stick: designing adoption in from the start. Your Lab 1 pre-work is assigned here.

One caution before we start, in the spirit of the course. The evidence here is real and sourced, and the causal story in our primary case is genuinely contested by serious people — we will hold that contest open rather than resolve it artificially, because learning to reason inside a disputed record is part of the point.

The frame. One arc — integrate, automate, make intelligent, make agentic. Three threads — Customer, Operations & Supply Chain, Cash Management. Four archetypes for scale, one operating-model lens for reading any company. Week 0 set the floor; Week 1 built the backbone; Week 2 asks why a working backbone so often fails to pay — and answers with the delivery reality, the cure (obliterate then automate), the Four Lenses, two cases a generation apart, and the adoption work that makes redesign stick.

A Working System That Still Doesn't Pay: Two Kinds of Failure

Start by separating two failures that get carelessly lumped together, because the whole week turns on the distinction. An **implementation failure** is the obvious kind: the system does not work. It crashes, it loses data, it cannot go live, the project overruns and is abandoned. Everyone can see it, and everyone agrees it is a failure. A **failure to leverage** is quieter and far more common. The system works — it processes transactions, it stays up, it passes its tests — and yet the organization is no better off, or is worse off, than before. The money was spent, the technology functions, and the value never arrives.

This is not a rare edge case; by the numbers it is the normal outcome. Gartner projects that by 2027, more than seventy percent of recently implemented ERP initiatives will fail to fully meet their original business-case goals, and that as many as a quarter of those will fail catastrophically ([Gartner, Enterprise Resource Planning](#)). Read that definition of failure again — “fail to fully meet their original business-case goals.” That is not a description of software that crashed. It is the precise description of a system that runs and still does not pay. The dominant failure mode of the single most expensive project most organizations ever undertake is failure to leverage, and it is the subject of this week.

Why does the quiet failure happen? Because value does not live in the software; it lives in the process the software runs. If you take a broken, tangled, twenty-step process and automate it exactly as it is, you have not fixed it — you have made it run faster and cost more to change. You have, in the phrase this week's cure made famous, paved the cow path. The cows wandered; the path follows their wandering; pour concrete over it and you have a permanent, efficient route that still goes nowhere sensible. This is the precise reason a technically successful project can leave an organization carrying real, daily cost — exactly what we watched at Bobst in Week 1, where a correct migration eliminated double data entry at one seam and re-created it at another.

Notice how this reframes the manager's job. The instinct, when a process is slow or error-prone, is to ask which tool will speed it up. The leverage question asks something prior: should this process exist in this shape at all? Most of the managerial failures that precede a visible technical disaster are failures to ask that prior question — to treat a software

project as a process-redesign decision wearing a technology costume. The rest of this week is about asking the prior question well.

Two failures, not one. Implementation failure means the system does not work; everyone sees it. Failure to leverage means the system works and the value still does not come, because the underlying process was never redesigned. Gartner's own definition of ERP failure — not meeting the original business-case goals — is failure to leverage by another name, and it is the majority outcome. Automating a broken process buys you a faster broken process.

Why ERP Projects Are the Hardest Thing a Company Does

Before the cure, understand the terrain, because it explains why leverage failure is the rule rather than the exception. Replacing or consolidating an integration backbone is, for most organizations, the largest and most consequential project they will ever run — it touches finance, operations, procurement, HR, and the customer all at once, and it cannot be rehearsed at scale. That is why the failure numbers are what they are, and why the post-mortems almost never blame the software. Modern branded ERP platforms are, on the whole, fit for purpose; they would not survive a brutally competitive market otherwise. The root causes cluster instead in human and organizational choices — planning, leadership, requirements, adoption — which is the through-line of this entire week.

The first thing to unlearn is the fantasy of the single, total system. Companies do consolidate onto one ERP, and consolidation is often the right move — but they almost never replace everything with it. The realistic end-state is a strong core backbone that must **integrate** with specialized applications rather than absorb them: a warehouse-management system, a treasury platform, an industry-specific tool, the Series-B startup's Stripe and Shopify. This is the **legacy, multi-system reality**: real companies run several imperfectly-integrated systems in parallel, some of them old, and the integration work between them never fully ends. It is why the seams between systems — not the systems themselves — are where value leaks, and why “we'll just put it all on one platform” is the first optimistic assumption a serious diagnosis has to retire.

The second thing to understand is who actually delivers a backbone, because the choice is consequential and often made carelessly. A real implementation is delivered by an ecosystem, not a purchase: systems integrators (the Accentures, EYs, and Capgeminis of the world), the company's own internal IT, the software vendor, and — decisively — the business stakeholders who own the processes being changed. Choosing a delivery partner is one of the highest-leverage decisions in the whole endeavor, and it rewards conscious, criteria-driven selection over the hasty or default choice. A word of caution the practitioners insist on: selecting purely on lowest price routinely costs more later, because

functionality or delivery expertise foregone at procurement resurfaces as overruns and rework during implementation. The partner decision is a business decision about capability and fit, not a line item to be minimized.

But the partner relationship carries a risk that runs the other way, and it is subtler than overpaying. Lean too hard on the integrator or the vendor — or simply accept the platform's predefined setup because it is the path of least resistance — and you quietly abdicate the understanding and the design choices the business itself should own. The vendor knows the software; only the business knows what its own processes are for. Retaining design authority — remaining the author of how your company works, even while others build the system — is as much a part of choosing a partner as the selection itself.

The third is how information gets into the backbone in the first place. A backbone earns its adoption partly by meeting people where they already work, through multiple **integration on-ramps** — web forms, email intake, file transfers, structured XML, and modern APIs. The more natural the on-ramps, the higher the adoption and the success rate, because people route their real work through the system instead of around it. Route work around the backbone and you have re-created, at a new seam, the very silos the backbone was meant to dissolve. All of this is why the recurring refrain of Module 1 holds: backbone work is a business transformation, not an IT project. The technology is the easy part. The organization is the hard part.

The terrain, not the tool. ERP is the largest, least-rehearsable project most firms run, and the software is rarely the root cause of failure — planning, requirements, partner choice, and adoption are. Consolidation is real but rarely total: a strong core still integrates with specialized apps, so the seams are where value leaks. A backbone is delivered by an ecosystem (integrators, internal IT, vendor, business owners), and choosing that partner consciously — not on lowest price — is a business decision, not a line item.

Requirements, and the Decisions You Can't Take Back Cheaply

If the software rarely fails and the organization usually does, where inside the organization does it fail? Very often, at requirements — the moment a company states what the system must actually do. Two kinds of requirement travel together and are worth separating. A **functional requirement** is a capability the business needs: being able to add a note to a purchase order, split a shipment, or approve a credit limit above a threshold. A **technical requirement** is how the system must be built or configured to deliver that: a data field, an interface, a nonstandard configuration, a custom-coded extension. The distinction matters because it is where a project quietly commits its future. Functional needs are

legitimate and must be met; the danger is meeting them through heavy custom builds and nonstandard configurations that then break, expensively, every time the vendor ships an update. The disciplined instinct is to prefer stable, standard configurations that survive upgrades, and to treat every proposed customization as a liability to be justified, not a convenience to be indulged.

Two failure modes haunt requirements specifically. The first is **mis-specification**: the requirement is captured wrongly or vaguely, so the built system solves a subtly different problem than the business has. The second is **knowledge loss over time**: the people who understood why a requirement existed leave, the reasoning is undocumented, and years later no one can safely change the configuration because no one remembers what it was for. Both are process-knowledge problems, not coding problems — and both are exactly what a good process map surfaces. This is the deep link between this week and Lab 1: mapping a process is how you discover its real requirements, functional and technical, and where they hide. Even a simplified map, done for the first time, makes visible what a hundred status meetings never quite pin down.

There is a timing law underneath all of this, and it is unforgiving. The decisions that most determine success — which suppliers and ecosystem to commit to, which partner to trust, what the requirements actually are — are made earliest, when they are cheapest to change, and they become dramatically more expensive to reverse as scope grows and the build hardens around them. A wrong early choice does not stay small; it compounds into costly rework. This is why enterprise transformation demands a different mindset from a routine IT project: in a routine project you can iterate cheaply; in a transformation, the expensive-to-reverse decisions come first, so you must think hardest at the beginning, when it is tempting to move fast and defer the hard questions.

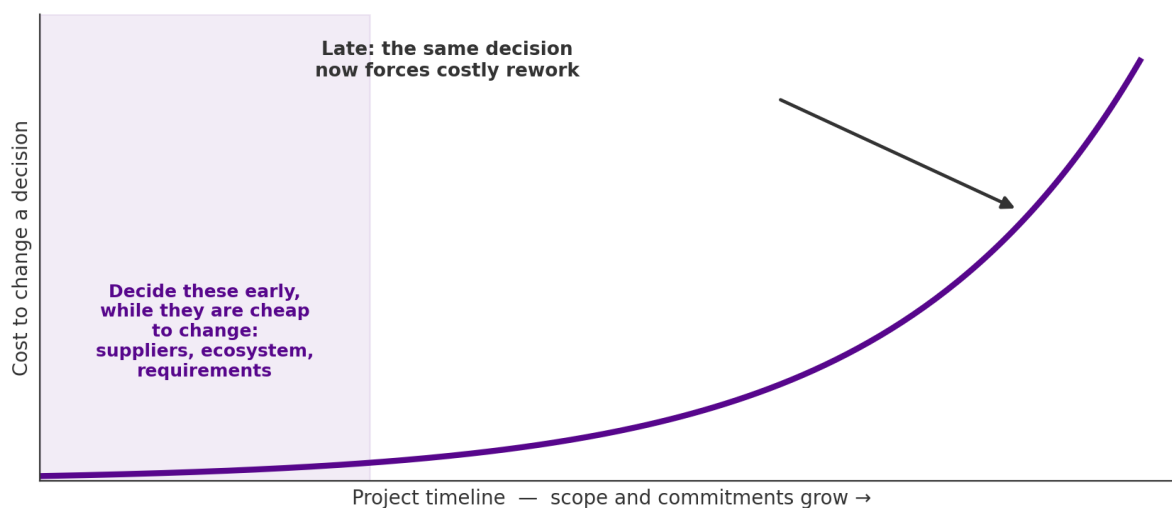


Figure 1. The cost of changing a decision rises steeply as scope and commitments grow, so the highest-stakes choices — suppliers, ecosystem, requirements — are the earliest ones. Adapted from Boehm's cost-of-change curve (Software Engineering Economics, 1981);

Decide the hard things early. Functional requirements (what the business needs) and technical requirements (how it must be built) travel together; meeting functional needs through heavy customization creates configurations that break on every update. Mis-specification and knowledge loss are process problems a map surfaces. And the decisions that most determine success — suppliers, ecosystem, requirements — are cheapest to change early and ruinous to reverse late. Transformation is not a routine IT project: think hardest at the start.

Don't Automate, Obliterate: Redesign Before You Build

So the terrain is unforgiving and the organization, not the software, is where projects fail. What is the cure? It was named in 1990 by Michael Hammer, in an argument this note teaches directly and whose origin you can read in his HBR essay if you want the full treatment (“Reengineering Work: Don’t Automate, Obliterate”). Hammer’s claim, written at the dawn of enterprise computing, was that companies were pouring money into information technology and getting disappointing returns because they were using the computer to mechanize old ways of working rather than to rethink them. His prescription was deliberately violent in its language: do not speed up the existing process; obliterate it, and redesign the work from a blank sheet around the outcome you actually want. He called the discipline business process re-engineering, and it is the cure to the disease we named at the top of the week.

Two of Hammer’s worked examples open his article, and every case in this course is a variation on them. Both turn on the same refusal — in Hammer’s words, to “stop paving the cow paths” — to obliterate the outdated process instead of embedding it in software.

The first is **Ford’s accounts-payable process**. In the early 1980s Ford’s North American accounts-payable department employed more than five hundred people, most of whose time went to reconciling mismatches — the moments when the purchase order, the receiving document, and the supplier’s invoice did not agree. Management’s first instinct was the reflexive one: install new systems to chase the mismatches faster, which they reckoned might trim the head count by about a fifth. Then they looked at Mazda, which ran the same function with just five people, and asked the deeper question — why are there mismatches at all? The redesign captured the purchase-order data once, at the source, in a shared database; when goods arrived, the receiving clerk checked them against the order on screen, and if they matched, the computer paid the vendor automatically — with no invoice at all, and only three data items to match instead of fourteen. The operating rule shifted from paying when the invoice arrived to paying when the goods arrived. Ford did not settle for its one-fifth; where it ran the new process it cut the function’s head count by seventy-five percent. The lesson is not “computers cut head

count.” It is that the saving came from eliminating the mismatch work, not from speeding it up.

The second is **Mutual Benefit Life**, an insurer whose application-approval process crawled through as many as thirty discrete steps, spanning five departments and nineteen people, each specialist touching the file and passing it on; a typical application took anywhere from five to twenty-five days, most of that time spent in transit between desks. Its president demanded a sixty-percent gain in productivity — far more than tinkering with the old process could deliver. Re-engineering swept away the departmental boundaries and created a single role: a case manager who owns an application from arrival to issued policy, working autonomously with a workstation, shared databases, and an expert system, and calling on a senior underwriter or physician only as an adviser. Turnaround fell from as long as twenty-five days to as little as four hours — two to five days on average — the company removed about a hundred field positions, and each case manager could handle more than twice the former volume. Same insight as Ford, different industry: the value came from redesigning who does the work and how it flows, not from automating the handoffs between the old silos.

Underneath both sits a principle this course returns to all semester, and it is deeper than headcount. The real thing Hammer wants obliterated is a **planning paradigm**. Most enterprises still run as batch-push systems: plans are computed overnight, cascade downstream, and humans spend the morning firefighting the exceptions. The precondition for automation — and later, intelligent automation and agents — to deliver real value is the shift toward pull-based, near-real-time, event-driven operations, where the system responds to demand signals as they happen and decisions are made at the point of variance rather than the morning after. Obliterate-then-automate is not only about removing steps; it is about replacing the rhythm the whole organization runs on. Keep that in your peripheral vision: it is the thread that connects this 1990 argument to the AI agents of Module 4.

The re-engineering move	What was obliterated, what replaced it, and why it paid
Ford — accounts payable	Obliterated: chasing purchase-order / receipt / invoice mismatches with armies of clerks. Replaced with: capture order data once in a shared database; pay automatically on a verified receipt, no invoice to reconcile. Why it paid — the saving came from eliminating the mismatch work, not automating it. Function headcount fell by roughly three-quarters.

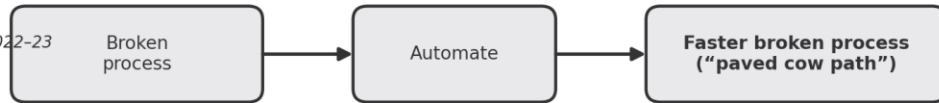
The re-engineering move	What was obliterated, what replaced it, and why it paid
Mutual Benefit Life — applications	Obliterated: a multi-step, multi-department relay of specialists handing a file along. Replaced with: one empowered case manager owning the application end to end, backed by a shared system. Why it paid — collapsing the handoffs cut processing time from days/weeks to a fraction and raised capacity.
The deeper target	Obliterated: the batch-push planning paradigm — overnight plans, morning firefighting. Replaced with: pull-based, event-driven operations where decisions happen at the point of variance. This is the redesign that makes automation, and later agents, actually pay.

One more question of sequencing decides whether a sound redesign survives contact with reality: how you roll it out. The tempting move is the **Big Bang** — cut the whole organization over to the new process and system on a single date. It is faster on paper and it is how both of this week's cases went live. But it converts every undiscovered problem into a simultaneous, enterprise-wide emergency. The alternative is a **progressive rollout** — phase the change by site, by module, or by process — which is slower but buys something precious: discovery. Each phase surfaces the mis-specifications and integration surprises while they are still contained, so the next phase is smarter and the blast radius of any single failure is bounded. Progressive rollout is not timidity; it is how you manage integration risk when, as we have seen, the expensive mistakes are the ones you cannot easily take back.

Obliterate, then automate — and phase it. Redesign the work from a blank sheet before automating it: Ford eliminated invoice mismatches rather than chasing them faster; Mutual Benefit Life replaced a departmental relay with one case manager. The deepest redesign is of the planning paradigm — batch-push to pull-based. And roll out progressively, not in a Big Bang: phasing buys discovery and bounds the blast radius. Sequence is everything: redesign first, automate second, phase throughout.

The trap

Nike 2000 · Birmingham 2022-23



The cure

— Hammer 1990



Figure 2. The same starting point, two paths: automate first and you pave the cow path; diagnose, obliterate, then automate, and you close the leverage gap. Adapted from Hammer, “Reengineering Work: Don’t Automate, Obliterate” (HBR, 1990);

Obliterate, but Selectively: Redesign Where Value Is Made or Lost

Hammer’s verb is deliberately violent, and taken too literally it becomes its own kind of trap. “Obliterate” is not a mandate to redesign everything. A team that reads it that way sets out to reinvent every process at once and fails for the opposite reason to Nike and Birmingham — not too little redesign, but too much of it, all at the same time, with no organization able to absorb the change. The discipline the word actually demands is selective. Decide, deliberately, where a process needs total redesign and where incremental improvement is enough, and concentrate the obliteration on the few areas where value is genuinely created or destroyed — leaving the parts that already work alone.

This is the other half of a thread you met in Week 1, where a packaged ERP and a custom-built backbone were set side by side and the choice between them treated as a strategic decision rather than a vendor default. The same discipline governs redesign. One failure is to reshape the business to fit whatever the platform offers out of the box, letting the software’s defaults decide how the company works; the opposite failure is to tear everything up at once. The disciplined path runs between them: redesign deliberately where the process drives the outcome, and adopt the standard where it does not. And selectivity is not only about focus — it is the first move in designing for adoption. You cannot carry an organization through the reinvention of everything it knows; concentrating redesign where it demonstrably pays is what makes the change survivable, and survivable change is the only kind that sticks. That adoption argument closes the week; hold the connection.

Obliterate selectively. “Obliterate” is not “redesign everything.” The discipline is to concentrate total redesign on the few areas where value is created or destroyed, adopt the standard where a process already works, and refuse both extremes — reshaping the whole business to fit the platform, or tearing up everything at once. Selectivity is also the first move in designing for adoption: focused change is change an organization can actually absorb.

Where Value Leaks: The Four Lenses in Full

If the cure is to obliterate selectively, you need a disciplined way to find the few areas worth obliterating — to locate where a process actually leaks value before you redraw it. You met the tool in Week 1 as a preview; here it is in full. The **Four Lenses** of process analysis (Dan Madison, 2005) are four different angles you hold up to any step, because each surfaces a kind of waste the others miss. A lens looks at a process, not at a database — you find the leak in the work first, then ask what data would prove it. (Madison’s book is an optional reference, not required reading, and is not held by NYU Libraries.)

The four are **Frustration, Time, Cost, and Quality**, and the discipline is to run all four over the same process rather than stopping at the first problem you notice. The Frustration lens is the human one: ask the people who actually live inside the process where it irritates them, because their daily aggravation is a reliable pointer to broken handoffs, dead ends, and rules that make no sense. It is also your earliest customer signal, since the friction employees feel is often the friction customers feel one step later. The Time lens hunts non-value-added steps — waiting, queuing, and especially rework, the time spent fixing what was not done right the first time. The Cost lens asks where money is consumed without producing value: duplicated effort, manual reconciliation, the standing army of people who exist only to make two systems agree. And the Quality lens looks for errors and defects, the failures that force the rework the Time lens found and the cost the Cost lens counts.

Lens	What it detects, where to look, and the thread it most often rings
Frustration	The human signal. Diagnosed by the people working inside the process; their irritation marks broken handoffs, dead ends, and senseless rules. Surfaces employee and, one step later, customer friction. Most often rings the Customer thread.
Time	Non-value-added steps: waiting, queuing, and above all rework. Map the end-to-end flow and ask where time is spent without advancing the outcome. Most often rings Operations & Supply Chain.

Lens	What it detects, where to look, and the thread it most often rings
Cost	Where money is consumed without producing value — duplicated effort, manual reconciliation, exception handling. Frequently the dollar translation of a Time or Quality problem. Most often rings Cash Management.
Quality	Errors and defects that force rework and erode trust. Quality problems propagate into Time and Cost, so a single quality fix often pays three times.

The lenses overlap on purpose, and the overlap is the most useful thing about them: a quality defect shows up again under Time (the rework it causes) and under Cost (the price of that rework), which tells you where one fix pays three times. Run them across the archetypes and they stay sharp. In the CPG manufacturer on SAP, a demand-planning miss is a Quality defect that becomes a Cost problem in obsolete inventory. In the mid-market manufacturer on NetSuite, an order that stalls between the sales desk and the shop floor is a Time leak first, felt as customer Frustration next, and booked as a Cash problem when the invoice slips a cycle. In the fifty-person services firm on HubSpot and Notion, a handoff from a signed proposal to a delivery plan that lives only in one person's head is a Frustration and a Quality risk that surfaces the day that person is on leave. In the Series-B startup stitching Stripe, Shopify, and QuickBooks, the founder re-keying orders between systems is a Frustration and a Time leak that will become a Cost problem the moment volume rises. Same four lenses, any scale.

This is not an academic exercise for this course — it is the engine of your Lab 1 pre-work, assigned this week. Each team picks a company and researches its real pain points and value leakages from public sources, and the Four Lenses are how you tag what you find: this is a Frustration problem, that is a Time problem, here is where Cost leaks, there is the Quality defect. The **½–1 page research brief** you submit before the Week 3 lab is, in effect, a Four-Lenses diagnosis of one company's process — the raw material you will map, pressure-test, and only then redesign, in the lab itself. Diagnose before you redesign; redesign before you automate.

The Four Lenses. Frustration, Time, Cost, Quality — four angles for finding where a process leaks value before you redraw it. Frustration is the human signal; Time hunts waiting and rework; Cost finds wasted money; Quality finds the defects that cause both. Run all four, not just the first; they overlap on purpose, so one fix can pay three times. This is the diagnostic engine of your Lab 1 research brief.

A Generation Ago: Nike, 2000

The trap we are about to see in our primary case is not new, so it helps to meet it first in a cleaner, older form. By the late 1990s Nike wanted speed: a way to forecast demand and plan production faster than its sprawling, multi-system order processes allowed. As part of a large supply-chain and ERP program, it deployed an i2 demand-and-supply-planning system to predict what the market would want and drive factory orders accordingly. The ambition was reasonable. The execution layered sophisticated planning software onto existing processes that had not been redesigned to receive it, with thin customization-and-testing and inadequate training of the planners who had to use it — and brought it live in a single aggressive push. The case is documented in Christopher Koch's account in CIO, "Nike Rebounds" (cio.com/.../nike-rebounds).

What happened is a small catalogue of the failure modes we have already named. The planning engine misjudged demand and, worse, its errors flowed straight through into real factory orders. It called for thousands too many of some shoes (the Air Garnett became the famous example) and thousands too few of others that were actually selling (the Air Jordans). The system ignored some orders, duplicated others, and deleted order data weeks after entry, so planners could not even reconstruct what they had told factories to make. As Koch's account makes clear, the software's problems would have stayed mere glitches had they not spilled into orders. The damage landed in the financials: Nike publicly attributed roughly \$100 million in lost sales to the episode, warned of a sharp profit shortfall, and saw its stock fall about twenty percent in a day — and the company's leadership pointedly measured that damage against the roughly \$400 million the broader supply-chain program had cost.

Read carefully, the failure was not in the ERP and not even, mainly, in the i2 software. It was the decision to automate demand planning on top of unredesigned processes — a forecasting engine treated as a crystal ball and bolted onto the old way of working, then rushed live in a Big Bang, without the training or testing that would have caught its errors before they reached a factory. Every theme of this week is present: no redesign before automation, a Big Bang rollout that turned a glitch into an enterprise event, and under-trained people who did not trust the tool. The recovery is the proof of the rule. Over the following year Nike slowed down, retrained its planners, and moved short- and medium-range sneaker planning off the predictive engine and into its SAP backbone, grounded in actual orders rather than a forecasting algorithm; in CIO, Nike's CIO Gordon Steele noted the move let the company "*simplify some of our integration requirements.*" Obliterate-then-automate, applied late but applied.

Thread. This is primarily an Operations and Supply Chain failure — demand planning, factory orders, inventory — with a sharp Cash overlay, since the whole cost surfaced as lost sales and a stock hit. Through the Four Lenses, it is a Quality failure (bad forecasts, mis-orders) that propagated straight into Cost.

Nike, 2000. A demand-planning engine was layered onto unredesigned processes, under-tested, and rushed live in a Big Bang with under-trained planners. It mis-ordered product straight into factories — too many Garnetts, too few Jordans — for roughly \$100M in lost sales and a ~20% stock drop. The ERP was not the villain; automating a broken process, all at once, was. Thread: Operations & Supply Chain, with a Cash overlay.

The Primary Case: Birmingham, 2022–23

Now the case to read closely, because it carries the official record and the sharpest version of the lesson. In April 2022, Birmingham City Council — the largest local authority in Europe — went live on a new Oracle Cloud Fusion system, replacing a long-standing SAP installation, to run its finance and HR. The original business case for the work was modest. But rather than re-engineering its finance and HR processes to fit a modern platform, the council attempted to bend the platform to its legacy ways of working, and it, too, went live in a cutover with known control weaknesses still unresolved. If Nike is the private-sector version of this mistake, Birmingham is the public-sector version, a generation later and at far higher stakes.

The cutover did not merely run slowly; it degraded the backbone's core function. The new system posted transactions incorrectly and could not produce reliable financial reports or reconcile the council's accounts without heavy manual intervention. In plain terms — and this is the whole reason the case belongs in Module 1 — the council lost its single source of truth. The one authoritative record of its own finances, the thing Week 1 said every function must be able to read and trust, was gone. Costs escalated far beyond the original case: the University of Sheffield's Audit Reform Lab ([full report, PDF](#)) documented an all-in cost that ran to many times that modest business case. On 5 September 2023, the council issued a Section 114 notice ([the council's statement](#)), the local-government equivalent of declaring it could not balance its budget.

Now the hard part, and the part the case is really here to teach: what actually caused the Section 114 is genuinely contested in the public record, and the contest is itself the lesson. The council's own official statement attributed the notice to its long-running equal-pay liability, together with an in-year budget gap; its statement did not name Oracle at all. The Sheffield Audit Reform Lab, reading the same public accounts, argues nearly the reverse: that the equal-pay figure was prematurely disclosed, unaudited, and used to deflect attention from the IT disaster, and concludes bluntly that *“Oracle seems the more likely explanation of Birmingham's problems.”* The external auditor took a middle line, treating the Oracle failure as a contributory factor and faulting governance and optimism rather than the core technology. More recently still, a group of finance and local-

government experts has argued the council may never have been truly bankrupt at all, on figures they call materially misstated; the commissioners reject that reading.

Do not resolve that dispute; understand why it exists. When an ERP cutover destroys the single source of truth, every downstream judgment degrades — including the judgment about what went wrong. The reason reasonable, expert people can still disagree years later about whether Oracle or equal pay caused the insolvency is that the failed system left the council unable to produce reliable numbers about its own position. That is the Week 1 thesis in its purest and most alarming form: lose the single source of truth and you lose not just operational control but the very ability to account for yourself. Frame Birmingham, then, not as “Oracle bankrupted a city,” but as this: a process-and-governance failure in an ERP cutover destroyed a public body’s capacity to know and prove its own finances, compounding a wider crisis — and the precise causal weight remains disputed precisely because of that destruction. Every theme of the week converges here: no redesign before the cutover, a Big Bang go-live, requirements and controls bent to legacy habits, and a transformation mistaken for an IT project.

Thread. Birmingham sits on Cash Management first — the literal inability to reconcile ledgers, forecast, and balance a budget — with a heavy Operations overlay across finance and HR. Through the Four Lenses it is the Quality failure (unreliable accounts) whose Cost consequences were, for a public body, existential.

Birmingham, 2022–23 — the primary case. An Oracle cutover that bent the platform to legacy processes destroyed the council’s ability to produce reliable accounts — its single source of truth — with costs escalating far beyond the original business case, and a Section 114 notice in September 2023. The causal weight of Oracle versus an unaudited equal-pay liability is genuinely disputed — which is itself the lesson: lose the single source of truth and you lose the ability to account for what went wrong. Thread: Cash Management, with an Operations overlay.

Two Cases a Generation Apart, One Trap

Set them side by side and the rhyme is exact. Nike, 2000: sophisticated planning software layered onto unredesigned supply-chain processes, rushed live on confidence, mis-ordering product into factories. Birmingham, 2022–23: a modern finance platform layered onto unredesigned finance and HR processes, bent to legacy habits, destroying the council’s books. Different decade, different sector, different technology — identical trap. In both, the organization treated a process-redesign problem as a software-installation problem, automated before it obliterated, went live in a Big Bang, and paid in the currency of the thread it touched: lost sales for Nike, an unbalanceable budget for Birmingham.

Read through the operating-model lens, both are stories of a Unification ambition without the Unification work. Recall the four stances from Week 1: Unification means high standardization and high integration — one process and one truth across the enterprise. As Ross and Weill develop the model in Chapter 2, they illustrate that both-high stance with Dow Chemical's globally integrated operations, and they are blunt that it is not free: greater standardization forces perfectly good local systems and ways of working to be ripped out and replaced by the common standard, a transition they call politically difficult and expensive. Both Nike and Birmingham were reaching for that destination, but neither did the re-engineering that earns it; each installed a system and hoped the old processes would conform. Nike kept its old planning behavior and bolted a forecasting engine onto it; Birmingham kept its legacy finance habits and bent a new platform around them. Each wanted the destination of Unification while skipping the standardization tax — the painful redesign of how work is actually done, and the adoption work that makes it stick — that buys the ticket.

Dimension	Nike (2000) vs. Birmingham (2022–23)
What was layered on what	Nike: an i2 demand-planning engine onto unredesigned supply-chain processes. Birmingham: an Oracle Cloud Fusion finance/HR platform onto unredesigned, legacy-bent processes.
How it was rolled out	Both: a Big Bang cutover that converted undiscovered problems into a simultaneous, enterprise-wide emergency — the opposite of a progressive, discovery-buying rollout.
What broke	Nike: factory orders — too many of the wrong shoes, too few of the right ones; order data lost. Birmingham: the books themselves — no reliable accounts, no reconciliation, no single source of truth.
What it cost	Nike: ~\$100M lost sales, ~20% stock drop. Birmingham: costs that ran to many times the original business case, and a Section 114 notice.
Operating-model read	Both: a Unification ambition (one standard, one truth) pursued without the re-engineering and adoption that earn it — the standardization tax skipped.
Primary thread	Nike: Operations & Supply Chain (Cash overlay). Birmingham: Cash Management (Operations overlay).
The identical trap. A generation apart, Nike and Birmingham made the same move: treat a process-redesign problem as a software install, automate before redesigning, and go live in a Big Bang. Both reached for a Unification-style single truth without paying the standardization tax that earns it. The cure was named in 1990 — obliterate, then automate, then phase it — and both learned it the expensive way.	

The Cure Has a Human Precondition: Adoption from the Start

There is a trap inside the cure, and you have to see it now or your redesigns will fail for a different reason than Nike's and Birmingham's did. Re-engineering a process changes how real people work — and people do not adopt a new way of working because a diagram says they should. The standard mistake is to treat the redesign as a technical project and then, near the end, bolt on a “change management” effort: a training session, a memo, a go-live date. That is too late. The discipline this course asks for is **adoption over change management**: design adoption into the redesign from the first day, not as a phase at the end.

Concretely, that means the people who live inside the process help diagnose it — the Frustration lens is your way in — help shape the redesign, and see their own friction addressed in it, so the new process is something done with them, not to them. It rhymes with everything else this week. It is why multiple integration on-ramps matter: they let people route real work through the backbone instead of around it. It is why the delivery ecosystem must include the business stakeholders who own the processes, not just integrators and IT. And it is the human face of the two cases: Nike rushed live with under-trained planners who did not trust the engine; Birmingham bent a platform to legacy habits no one had been brought along to abandon. Recall, too, the European packaging company from Week 1 — the very case Ross and Weill use in Chapter 2 for a Diversification-to-Unification shift. Moving every country onto one central ERP with a single product and price list cut a change that once meant updating fifteen systems over weeks down to a matter of hours — real gains — but it stripped country managers of their authority over products, pricing, and promotions, and, in Ross and Weill's own telling, they naturally resisted. That resistance is the standardization tax made flesh. Westerman's *Leading Digital* makes the same point with data in Chapter 1: firms that pour investment into technology but lack the leadership to drive change through the organization — his “Fashionistas” — gain revenue efficiency yet are no more profitable than their peers, because technology pays only when leadership turns it into transformation. Redesign is a people problem before it is a process problem, and adoption is how you keep the leverage you re-engineered.

Adoption, not afterthought. Re-engineering changes how people work, so adoption must be designed in from the start — the people inside the process help diagnose and reshape it — rather than bolted on as end-stage “change management.” It is the human reason Nike, Birmingham, and even Week 1's packaging firm paid a price, and the reason integration on-ramps and stakeholder ownership matter. Redesign is a people problem before it is a process problem.

Where This Week Sits in the Course

The threads. Week 2 is where you learn to hear a failure as a thread, and to name it with a lens. Nike rang the Operations and Supply Chain thread (with a Cash overlay); Birmingham rang Cash Management (with an Operations overlay). Reading which thread a failure lands on, and which of the Four Lenses surfaces it, is half of diagnosing it — and it is exactly the move your capstone will ask you to make for your own company.

The labs. This week carries a deliverable: Lab 1 is a two-part exercise spanning Weeks 2–3, and its pre-work is assigned now. Pick your company, research its real pain points and value leakages from public sources through the Four Lenses, and draft a ½–1 page research brief — submitted and checked before the Week 3 lab, because the in-class build assumes you arrive with a diagnosis in hand. Carry this week’s discipline into it. Before you label any step a candidate for automation, ask Hammer’s question: should this step exist in this shape at all, or is it a cow path waiting to be obliterated? And when you map the process in the lab, notice that the map is also surfacing requirements — the functional needs and the technical demands — and the control points where data is born and the decision points where work is routed, approved, or rejected; that is the same diagnosis a real implementation team runs, in miniature.

The capstone. The board memo you build all term lives or dies on this week’s distinction. A board does not fund “buy this software.” It funds a redesigned process that the software enables — a leverage case, not an installation case. This week’s homework artifact is a one-page context-and-diagnosis brief on your company’s current backbone and where it fails to leverage, which becomes Section 2 of your board memo (Context and Background); your team charter is also due at the end of this week. Section 2, with the operating-model read you began in Week 1, is the ground on which every later recommendation rests.

Your Study Questions

Bring rough answers to our session; these are meant to be argued, not recited.

1. What managerial failures most commonly precede a technical integration failure?

Look past the code. The post-mortems rarely blame the software — modern ERP platforms are fit for purpose — and Gartner’s own account puts the usual causes in planning, requirements, leadership, and adoption. In both cases the technical failure was downstream of managerial choices: treating a process-redesign decision as a software purchase, going live in a Big Bang on confidence instead of phasing and testing, skipping the redesign that would have made the system fit, bending requirements and controls to legacy habits, choosing partners or scope carelessly and early, and failing to bring along the people who run the process. The technical failure

is usually the visible tail of a managerial failure to ask whether the process should exist in its current shape at all.

2. Define 'failure to leverage' in contrast to mere implementation failure.

An implementation failure is a system that does not work — visible, agreed, and comparatively rare. A failure to leverage is a system that works while the value never arrives, because the underlying process was automated rather than redesigned; Gartner's own definition of ERP failure — not meeting the original business-case goals — is exactly this, and it is the majority outcome. Use Birmingham to complicate the line usefully: an implementation can fail so badly (no reliable accounts) that it also becomes the ultimate failure to leverage — the organization cannot even measure the value it was supposed to gain. Nike, by contrast, is a cleaner failure-to-leverage story once you separate the working ERP from the mis-used planning layer bolted onto an unredesigned process.

3. Why must BPR precede automation — and what human risks are introduced when it is skipped?

Because automation freezes whatever process it runs; if you freeze a broken process you get a faster, costlier-to-change broken process — the paved cow path. Re-engineering first removes the brokenness so that automation amplifies something worth amplifying (Ford eliminated mismatches; Mutual Benefit Life collapsed a relay). The human risks of skipping it run in two directions: skip redesign and you institutionalize dysfunction and the daily toll of people moving data between systems; but redesign also carries a human cost of its own — the standardization tax, the autonomy and roles it strips from people, the resistance the European packaging company's country managers showed in Week 1. That is exactly why adoption must be designed in from the start, and why a progressive rollout, which gives people time to absorb the change, usually beats a Big Bang. The point is not that redesign is painless, but that automating without it pays the human cost and gets none of the benefit.

Running Glossary

Failure to leverage — when a system works yet the expected value never arrives, because the underlying process was automated rather than redesigned. Gartner's definition of ERP failure — not meeting the original business-case goals — is this by another name. Contrast with implementation failure (the system does not work).

Selective redesign — deciding where a process needs total redesign and where incremental change suffices, and concentrating the redesign on the few areas where

value is created or destroyed rather than reshaping the whole business to fit the platform. The disciplined reading of Hammer's "obliterate."

Retained design authority — remaining the author of how your own business works even while integrators or a vendor build the system. The failure mode is ceding that authority to the vendor or to the platform's predefined setup.

Decision points — the places in a process where work is routed — approved, rejected, or sent one way rather than another — as distinct from control points, where data is first created or a commitment is made. Rejection and rework are the costly pattern. (Introduced in Week 1.)

Business process re-engineering (BPR) — redesigning a process from a blank sheet around the outcome you want, rather than mechanizing the existing one (origin: Hammer, 1990).

Obliterate, then automate — the sequencing rule: remove and redesign the broken process before applying technology to it.

Paving the cow path — automating an existing, unredesigned process — making a wandering route fast and permanent instead of straightening it.

The Four Lenses — four angles for diagnosing where a process leaks value: Frustration (the human signal), Time (waiting and rework), Cost (wasted money), Quality (errors and defects). Run all four; they overlap.

Legacy / multi-system reality — the normal end-state in which a company runs a core backbone plus several imperfectly-integrated specialized systems in parallel, so integration work never fully ends and the seams are where value leaks.

Delivery ecosystem — the parties who deliver a backbone — systems integrators, internal IT, the software vendor, and the business stakeholders who own the processes — among whom choosing a delivery partner is a high-stakes, conscious decision.

Integration on-ramps — the ways work enters the backbone — forms, email, file transfer, XML, APIs. More natural on-ramps raise adoption; missing ones push work around the system and re-create silos.

Functional vs. technical requirements — functional = what the business needs the system to do (e.g., add a note to a purchase order); technical = how it must be built or configured to do it (e.g., a nonstandard configuration). Heavy customization to meet functional needs creates configurations that break on updates.

Big Bang vs. progressive rollout — Big Bang cuts the whole organization over at once (fast, but turns every hidden problem into a simultaneous emergency); a progressive rollout phases the change to buy discovery and bound risk.

Batch-push vs. pull-based / event-driven — the planning paradigm BPR ultimately targets: from overnight plans cascaded downstream with morning firefighting, to operations that respond to demand signals at the point of variance.

Adoption (over change management) — designing the people-side of a redesign in from the start — those inside the process help diagnose and reshape it — rather than bolting change management on at the end.

Section 114 notice — a UK local-authority report declaring it cannot balance its budget; effectively a municipal declaration of insolvency. Birmingham issued one on 5 September 2023.

The Shift in Mindset

If you take one thing from this week, let it be the order of operations. The reflex — “this process is slow, what should we automate it with?” — is the reflex that cost Nike \$100 million and Birmingham its books. Replace it with the sequence this week has built: understand the terrain (integration is a transformation, not an IT project, and its expensive decisions come early); diagnose with the Four Lenses; obliterate before you automate; roll out progressively rather than all at once; and design adoption in the whole way, so the redesign sticks. The backbone you built in Week 1 is necessary; redesigning the work that runs on it is what turns it from cost into value, and closes the leverage gap. Next week you stop reading other people’s failures and start mapping your own company’s process — and the first thing you will look for is the cow path.